

CAHIER DES CHARGES

Projet d'Ingénierie Big Data & Analyse de Graphes Temps Réel

Plateforme de Streaming Infini d'Interactions Commerciales (Style "LeBonCoin")

Traitement Flux Continu, Pipelines PySpark et Visualisation Dynamique de Graphes

Table des matières

1	Présentation Générale et Contexte	2
2	Spécifications Fonctionnelles	2
2.1	Génération du Flux de Données Événementielles (Streaming Infini)	2
2.2	Visualisation et Rafraîchissement Graphique Dynamique	2
3	Architecture Technique et Implémentation PySpark	3
3.1	Concepts Clés PySpark requis	3
3.2	Schéma logique du Pipeline de données	3
4	Livrables attendus et Évaluation	3

1 Présentation Générale et Contexte

Ce projet s'inscrit dans le cadre du module d'architecture et de programmation distribuée Big Data. L'objectif est de concevoir, implémenter et évaluer une architecture applicative capable de gérer un flux de données massif et infini simulant les interactions des utilisateurs d'une plateforme type "LeBonCoin").

La plateforme génère en continu des actions utilisateur complexes sur des produits mis en vente par des annonceurs. Le point central du projet consiste à interconnecter ces entités (**Utilisateurs, Vendeurs, Produits**) sous la forme d'un réseau interconnecté modélisé par un graphe de connexions évolutif. Ce graphe doit être rafraîchi de manière incrémentale et visualisé dynamiquement au fil de l'eau.

2 Spécifications Fonctionnelles

2.1 Génération du Flux de Données Événementielles (Streaming Infini)

Le système doit intégrer un simulateur autonome (producteur) capable de générer un flux ininterrompu d'événements au format JSON. Chaque événement caractérise une interaction précise sur le site. Les trois actions fondamentales attendues sont :

- **AIME (Like)** : Manifestation d'un intérêt préliminaire d'un utilisateur pour un produit donné.
- **VOUT VOULOIR ACHETER (Intent)** : Action forte indiquant l'ajout à un panier, la prise de contact avec le vendeur ou une intention d'achat marquée.
- **ACHAT (Purchase)** : Finalisation de la transaction financière fermant le cycle de l'article (et pouvant déclencher sa désactivation).

Chaque enregistrement du flux de streaming doit obligatoirement comporter les champs structurels suivants :

Champ	Type	Description	Exemple
timestamp	Timestamp	Heure de l'événement (ISO 8601)	"2026-05-25T09:15:30Z"
user_id	String	Identifiant unique de l'acheteur potentiel	"usr_9482"
user_city	String	Localisation de l'utilisateur	"Paris"
product_id	String	Identifiant unique de l'article en vente	"prod_5501"
product_cat	String	Catégorie de l'objet	"Véhicules"
seller_id	String	Identifiant de l'annonceur / vendeur	"sel_0214"
action_type	String	Nature de l'action (AIME, VOUT, ACHAT)	"VOUT"
price	Double	Valeur marchande associée à l'article	450.00

2.2 Visualisation et Rafraîchissement Graphique Dynamique

L'application doit proposer un tableau de bord (Dashboard) ou une interface graphique présentant une vue dynamique sous forme de **Graphe de Connexions**.

1. **Les Nœuds (Sommets)** : Doivent représenter visuellement de manière distincte (par une couleur ou un symbole) les entités : Utilisateurs (*U*), Vendeurs (*S*), et Produits (*P*).
2. **Les Liens (Arêtes)** : Représentent les interactions pondérées ou typées orientées (ex. *UAIMEP* ou *SPROPOSEP*).
3. **Politique de Rafraîchissement** : L'affichage graphique du graphe de relations doit se mettre à jour automatiquement selon une fréquence paramétrable (ex. toutes les 5 secondes ou à chaque micro-batch Spark), reflétant l'évolution des degrés des nœuds et les nouvelles connexions tissées par le flux.

3 Architecture Technique et Implémentation PySpark

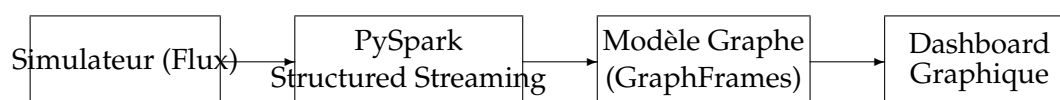
L'architecture s'appuie sur l'écosystème Apache Spark couplé à Python (PySpark). Les étudiants devront obligatoirement mettre en œuvre et expliciter les concepts technologiques suivants :

3.1 Concepts Clés PySpark requis

- **SparkSession & SparkContext** : Initialisation du point d'entrée unique de l'application avec configuration optimisée de la mémoire et des parallélismes de shuffle (`spark.sql.shuffle.par`).
- **PySpark Structured Streaming** : Utilisation des DataFrames de streaming pour consommer le flux infini. Définition stricte du schéma de lecture (*Schema Enforcement*) pour éviter l'inférence coûteuse.
- **Gestion du Temps et Fenêtrage (Windowing)** : Implémentation de fenêtres de temps glissantes (*Sliding Windows*) ou tumultueuses (*Tumbling Windows*) pour agréger les volumes d'actions (AIME, VOUT, ACHAT) par tranches temporelles.
- **Gestion des Retards (Watermarking)** : Utilisation obligatoire de `withWatermark()` afin de gérer les données arrivant en retard dans le flux streaming et permettre le nettoyage automatique des anciens états en mémoire.
- **Modes de Sortie (Output Modes)** : Justification et mise en application du mode adéquat (Append, Update, ou Complete) selon la nature des agrégations et de la mécanique d'affichage choisie.
- **Modélisation de Graphes (GraphFrames / GraphX)** : Utilisation de la bibliothèque **GraphFrames** pour manipuler le graphe de connexions constitué du DataFrame des sommets (Vertices : `id, type, label`) et du DataFrame des arêtes (Edges : `src, dst, relationship`). Calcul d'indicateurs de centralité ou de composants connectés au fil de l'eau.

3.2 Schéma logique du Pipeline de données

Le pipeline de traitement doit respecter la topologie d'interconnexion suivante :



4 Livrables attendus et Évaluation

Le projet donnera lieu à la production de trois livrables majeurs soumis à évaluation :

1. **Le Code Source** : Un dépôt Git propre comprenant le script du générateur de données, le script principal PySpark de traitement de flux et le code de l'interface graphique de rafraîchissement.
2. **Le Rapport Technique** : Un document synthétique explicitant l'architecture logicielle retenue, la justification des configurations de fenêtrage et de watermarking PySpark, ainsi que les métriques de performance du traitement de flux.